

Creating Web Services Applications with IntelliJ IDEA

In this tutorial you will:

1. Create IntelliJ IDEA projects for both client and server-side Web Service parts
2. Learn how to tie them together
3. Use various frameworks and technologies: Apache Axis, JAX-WS, RESTful Web Services
4. Experience advanced IntelliJ IDEA coding assistance and code generation features

Prerequisites

To develop Web Services applications with IntelliJ IDEA, download the following software:

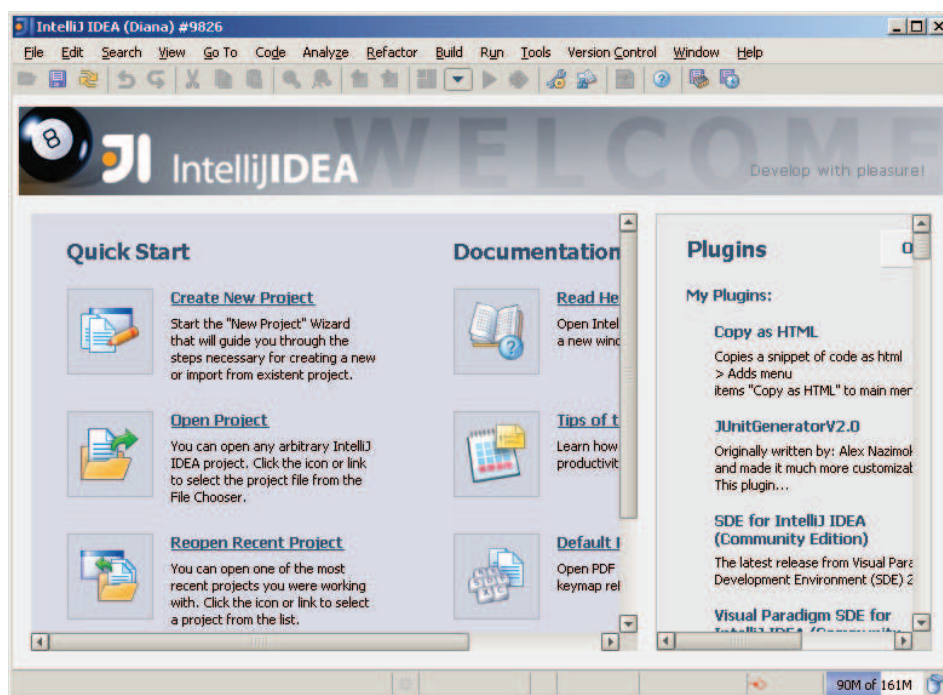
1. IntelliJ IDEA 8.1 or later release build. You can get it from <http://www.jetbrains.com/idea>
2. A compatible application server, for example, Apache Tomcat, available at <http://tomcat.apache.org/>

You may also want to check <http://www.jetbrains.com/idea/training/demos.html> and <http://www.jetbrains.com/idea/documentation/documentation.html> to get better insight into IntelliJ IDEA and its features.

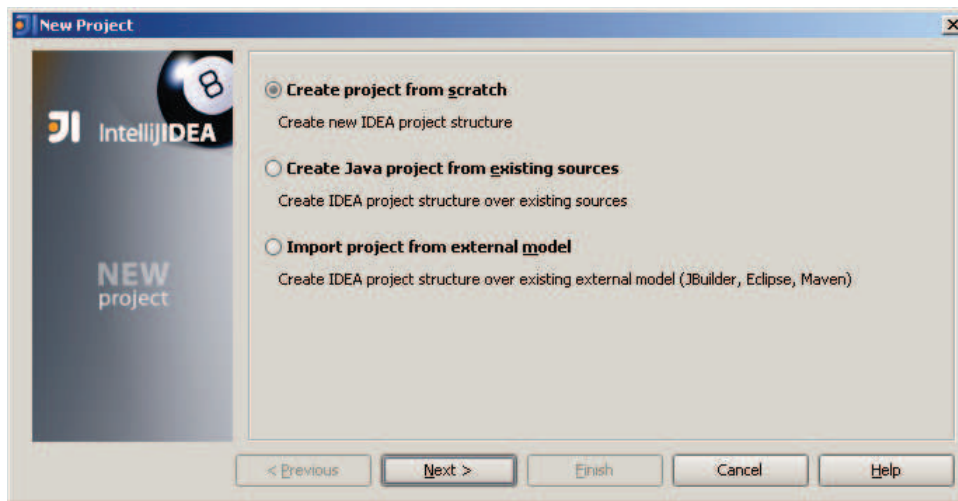
Creating Web Service: Apache Axis

Prior to proceeding with the tutorial steps, make sure that an application server is installed on your machine. After that, launch IntelliJ IDEA and begin with creating a project from scratch.

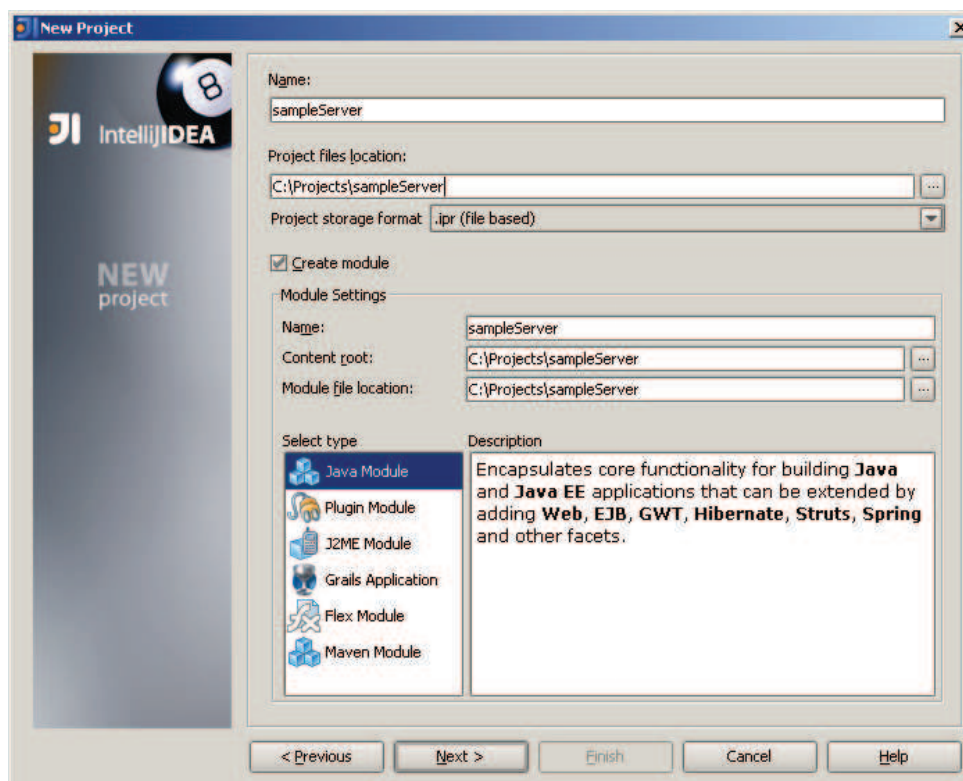
1. Run IntelliJ IDEA.



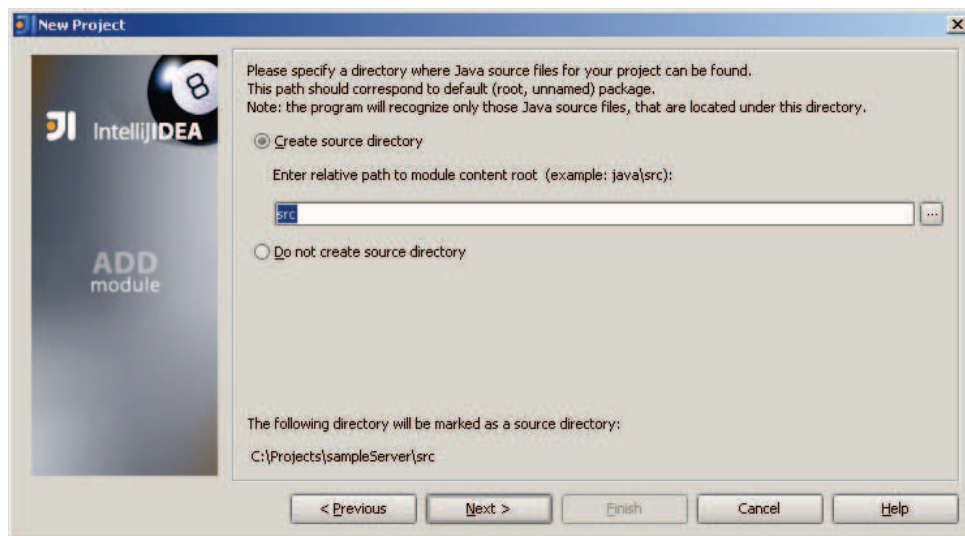
- On the **File** menu, click **New Project**. The **New Project** wizard appears. Click **Next**.



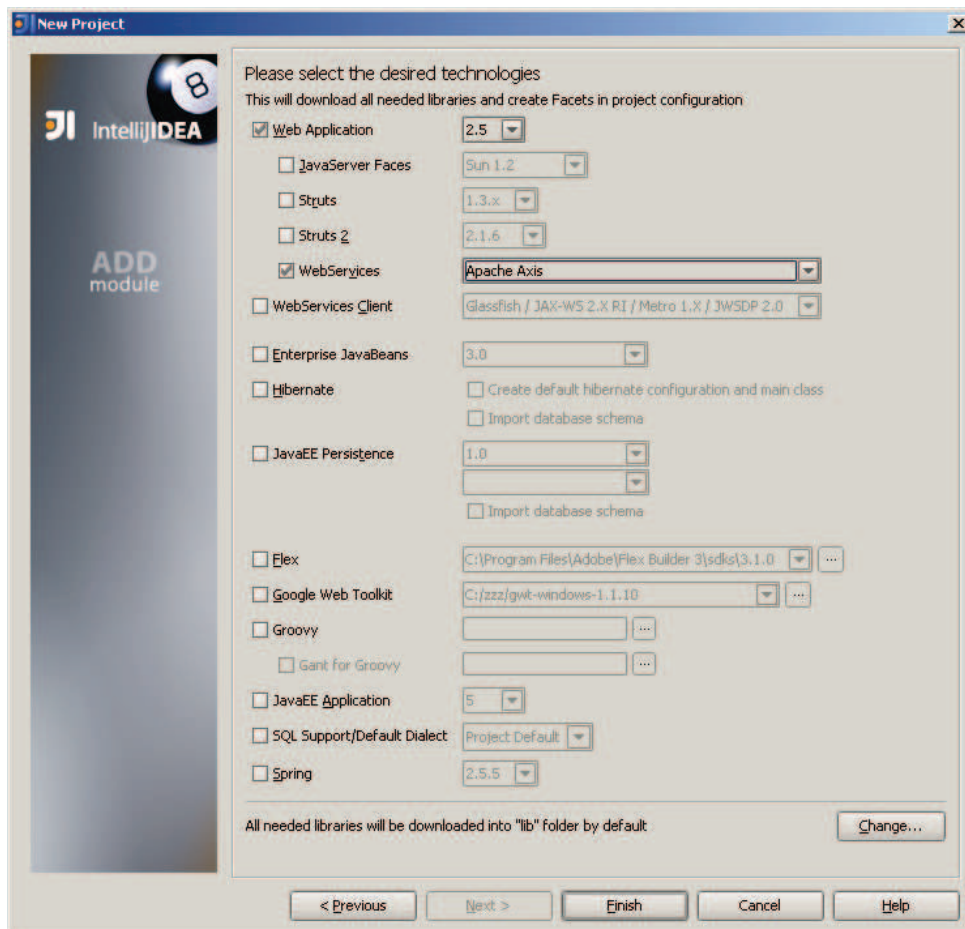
- Specify the project name, for example, **sampleServer**.



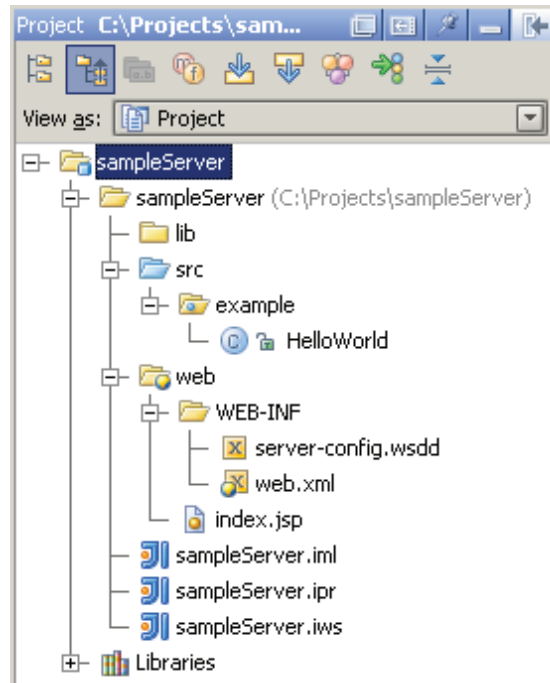
4. Leave the option to create source directory selected and click **Next**.



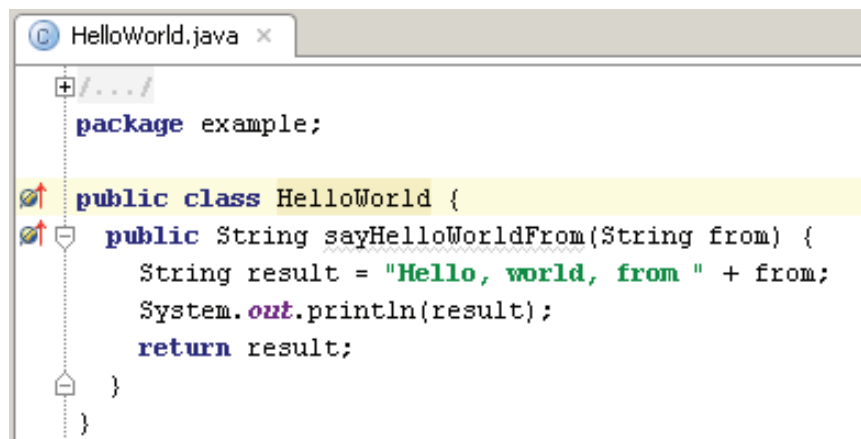
5. From the list of technologies, select **Web Application**, then **WebServices**, and select **Apache Axis** from the list. Click **Finish**.



Now, IntelliJ IDEA generates the project with some sample code — **HelloWorld** class. This is how the project structure should look.



Sample **HelloWorld** class contains the generated code that concatenates a given string with **Hello, world, from** and then sends it back to the client along with printing to the server console.



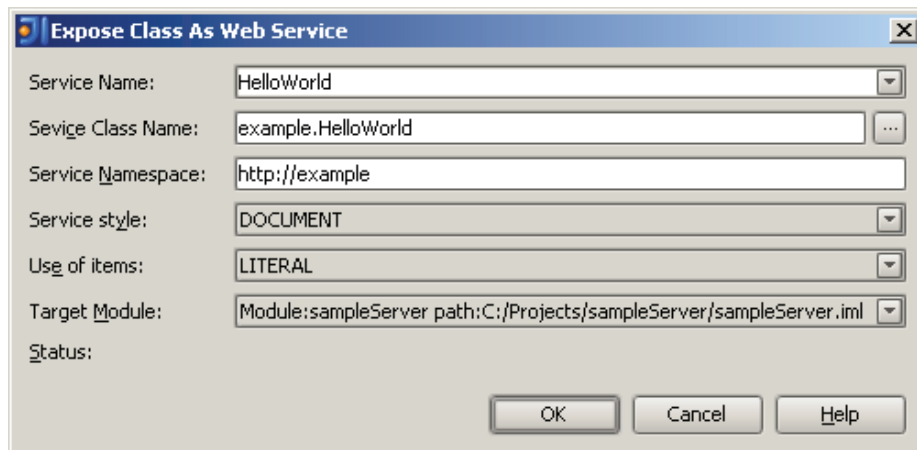
We only need to mark this class as one implementing a Web service by adding the **@WebService** annotation. IntelliJ IDEA suggests a quick-fix for it. Click the light bulb, or press **Alt + Enter** to automatically add annotation and corresponding import.

Another quick-fix we need to apply is adding **@WebMethod** annotation. We'll need the **sayHelloWorldFrom** method annotated, as we're going to expose it as a web service operation.

When done, press **Ctrl+Shift+F9** to compile the HelloWorld class.

We've only few things left to do: expose the class as Web Service and generate the appropriate WSDL descriptor file.

1. Select the class name in the editor
2. Select **Tools | Web Services | Expose Class as Web Service** menu.

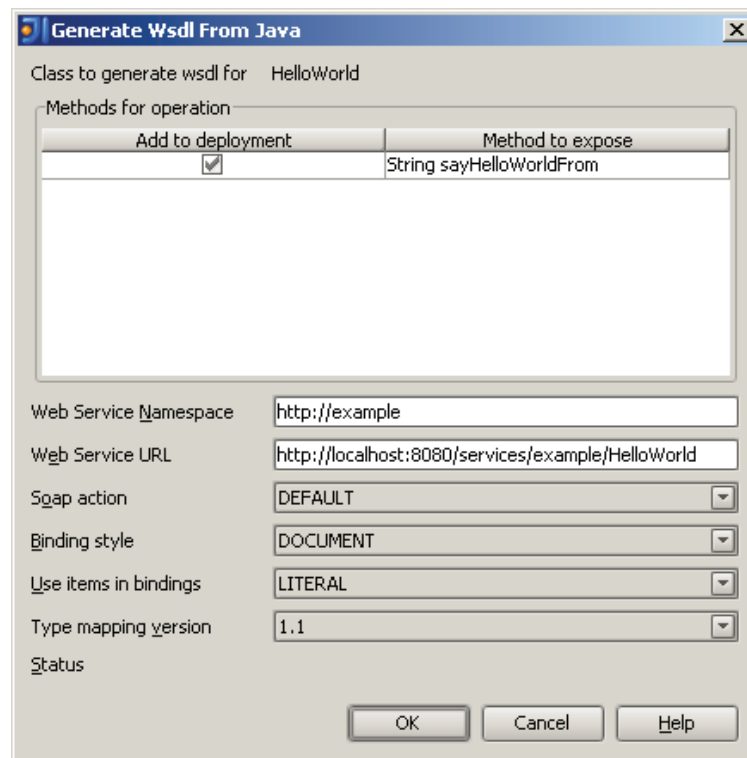


3. Click **OK**. IntelliJ IDEA automatically adds service description to the **server-config.wsdd** file:

```
<service name="HelloWorld" provider="java:RPC" style="document"
use="literal">
    <parameter name="className" value="example.HelloWorld"/>
    <parameter name="allowedMethods" value="*" />
    <parameter name="scope" value="Application" />
    <namespace>http://example</namespace>
</service>
```

IntelliJ IDEA is also capable of generating WSDL descriptors directly from the Java code.

1. Select the class name in the editor
2. Select **Tools | Web Services | Generate Wsdl From Java Code** menu



3. All we need to do is click **OK**. The descriptor is generated automatically.

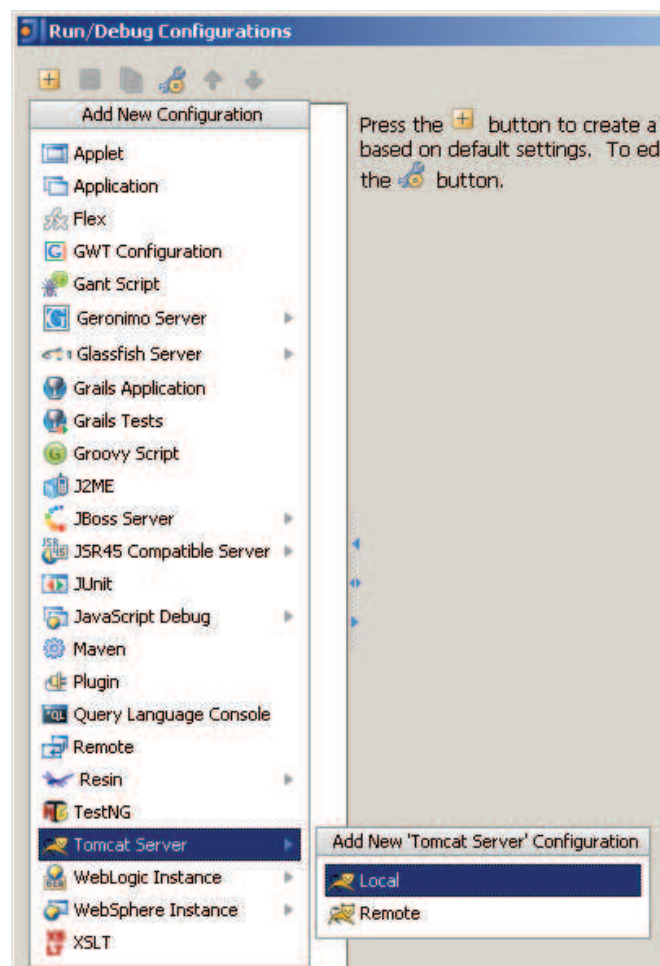
Note. **Web Service URL** field shows you the URL at which this Web Service will be available. You will need this URL to generate client code later in this tutorial; the value can be found in the generated WSDL file:

```
<wsdl:soap:address location="http://localhost:8080/services/example/HelloWorld"/>
```

You can find all generated files in the project tree and edit them manually, if required. IntelliJ IDEA provides full coding assistance, including WSDL/WADL-aware code completion, plus inspections and quick-fixes, and even refactoring.

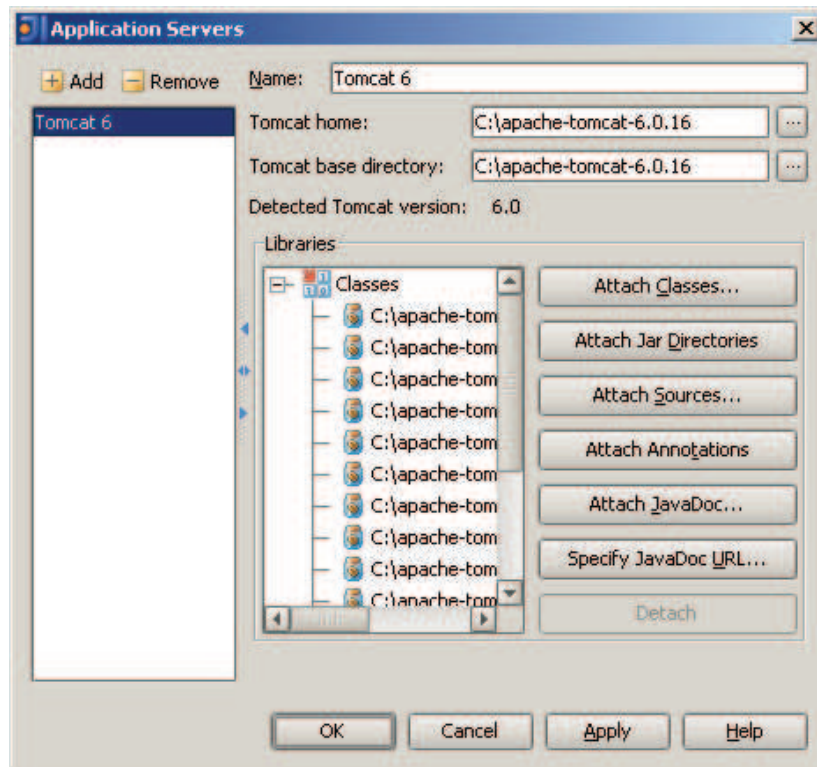
Let's deploy the Web Service to make sure everything works as expected. For that, we need to create an application server **Run/Debug Configuration**.

1. On the main menu, select **Run** and then click **Edit Configurations**.
2. Click **plus** button to add a configuration. As we already mentioned, we're using Tomcat server.

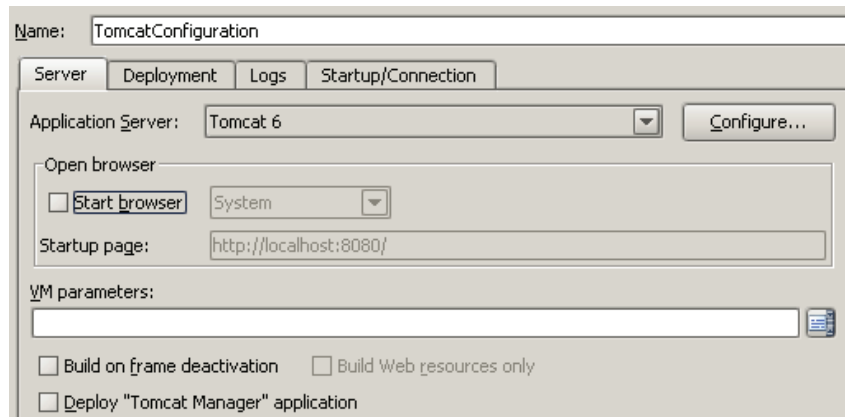


3. Specify the configuration name, for example, **TomcatConfiguration**.

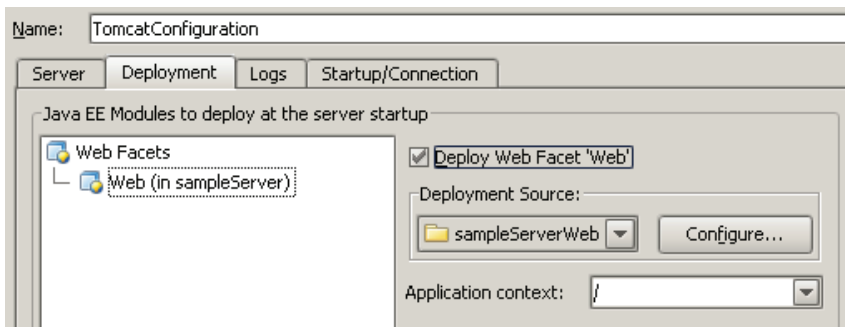
4. In Server tab, click **Configure. Application Servers** dialog appears. Click **plus** button to add the server configuration. In the **Tomcat home** field specify the folder where you have installed the server.



- Click **OK**. Back in the **Run/Debug Configuration** dialog box, and then select the configured server from the **Application Server** list. Make sure the **Start browser** option is clear — we don't need a Web browser launched for now.

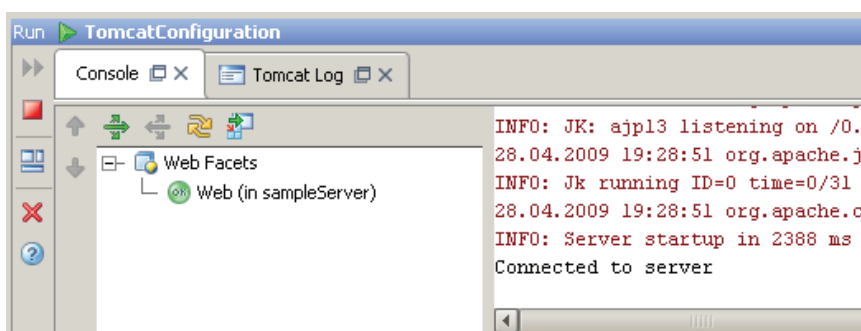


- Click **Deployment** tab. Select the web facet corresponding to **sampleServer** (notice that the facet has been automatically configured), and select the **Deploy Web Facet 'Web'** option



- Click **OK** to save the configuration.

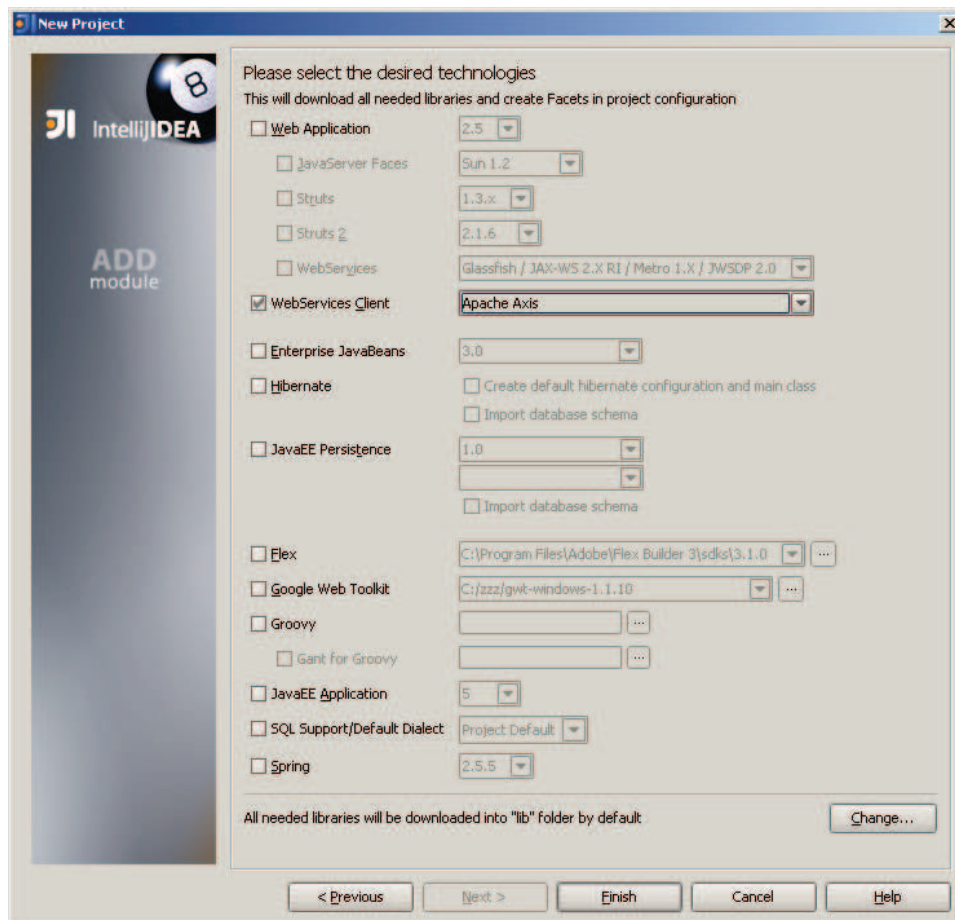
Now everything is ready to run, so just make sure the configuration is selected in the toolbar and press **SHIFT+F10**. IntelliJ IDEA compiles, deploys and runs the application.



Our Web Service is now running and we're ready to move forward.

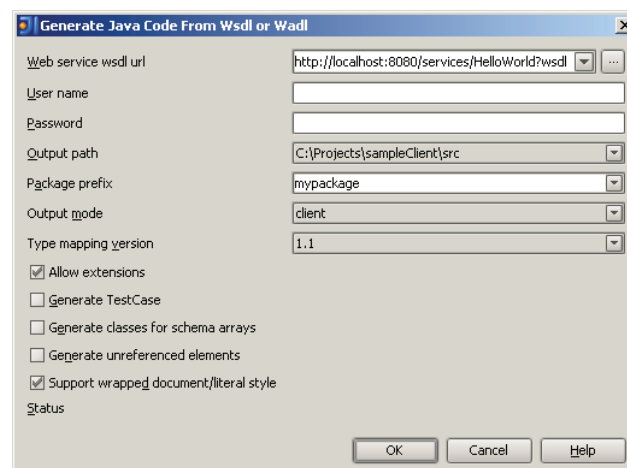
Creating Web Service Client: Apache Axis

We need to create another project, **sampleClient**. It's essentially the same procedure as we've used in the beginning of this tutorial. The only difference is that we have another selection of supported technologies. Namely, select the **Web Services Client** option and then select **Apache Axis** from the list.



When you click **Finish**, IntelliJ IDEA asks you whether you want to open new project in a new frame or current one. Click **New Frame**.

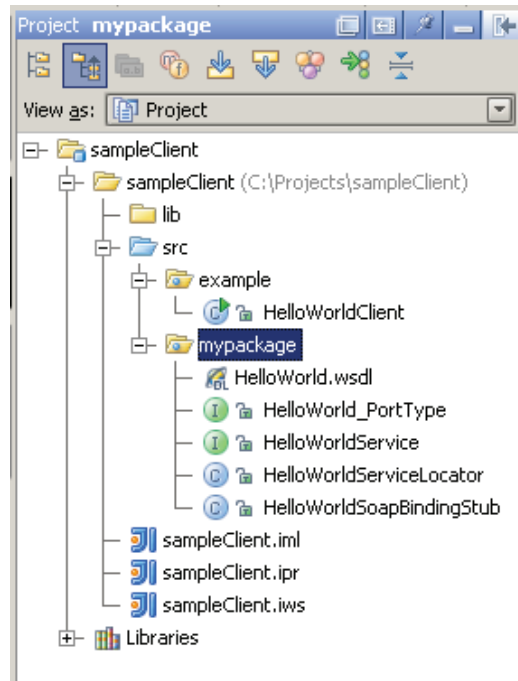
After the project is created, the following dialog pops up.



All we need to do here is specify the **Web Service URL** and click OK.

Note: In general case, this URL depends on the container you use to deploy your web service, and can be retrieved from the deployment descriptor. In our case the url is **`http://localhost:8080/services/HelloWorld?wsdl`**.

IntelliJ IDEA automatically generates the required code and project structure.



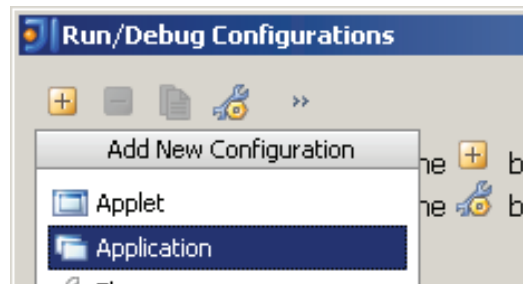
Code editor opens with **HelloWorldClient** class from **example** package, which contains Live Template, with several fields where you need to specify a value by selecting one of choices offered by IntelliJ IDEA. After that, add a declaration for variable that will contain service **response**, plus the **println** call to display it in the console.

```
package example;

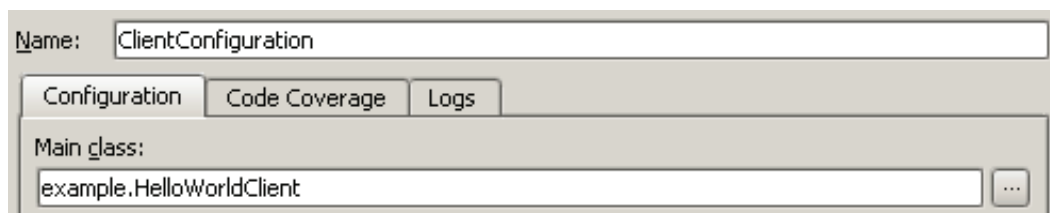
import mypackage>HelloWorld_PortType;
import mypackage>HelloWorldServiceLocator;

public class HelloWorldClient {
    public static void main(String[] argv) {
        try {
            HelloWorldServiceLocator locator = new HelloWorldServiceLocator();
            HelloWorld_PortType service = locator.getHelloWorld();
            // If authorization is required
            //((HelloWorldSoapBindingStub)service).setUsername("user3");
            //((HelloWorldSoapBindingStub)service).setPassword("pass3");
            // invoke business method
            String response = service.sayHelloWorldFrom("John");
            System.out.println(response);
        } catch (javax.xml.rpc.ServiceException ex) {
            ex.printStackTrace();
        }
        catch (java.rmi.RemoteException ex) {
            ex.printStackTrace();
        }
    }
}
```

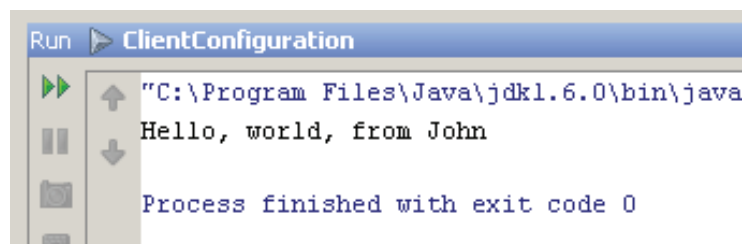
To run the client application, create a standard Java Application run configuration.



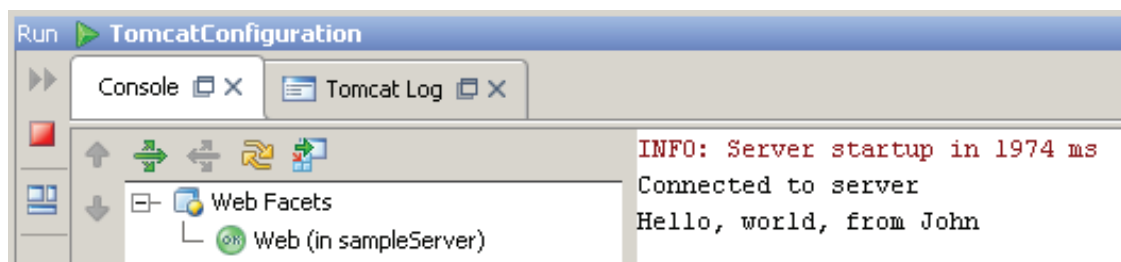
The only option we need to configure is to specify **example.HelloWorldClient** as main class.



Save configuration and press **SHIFT+F10** to run it. Application will print the output to console.



In the meantime, we can examine the server console to see that the same string has been printed there.



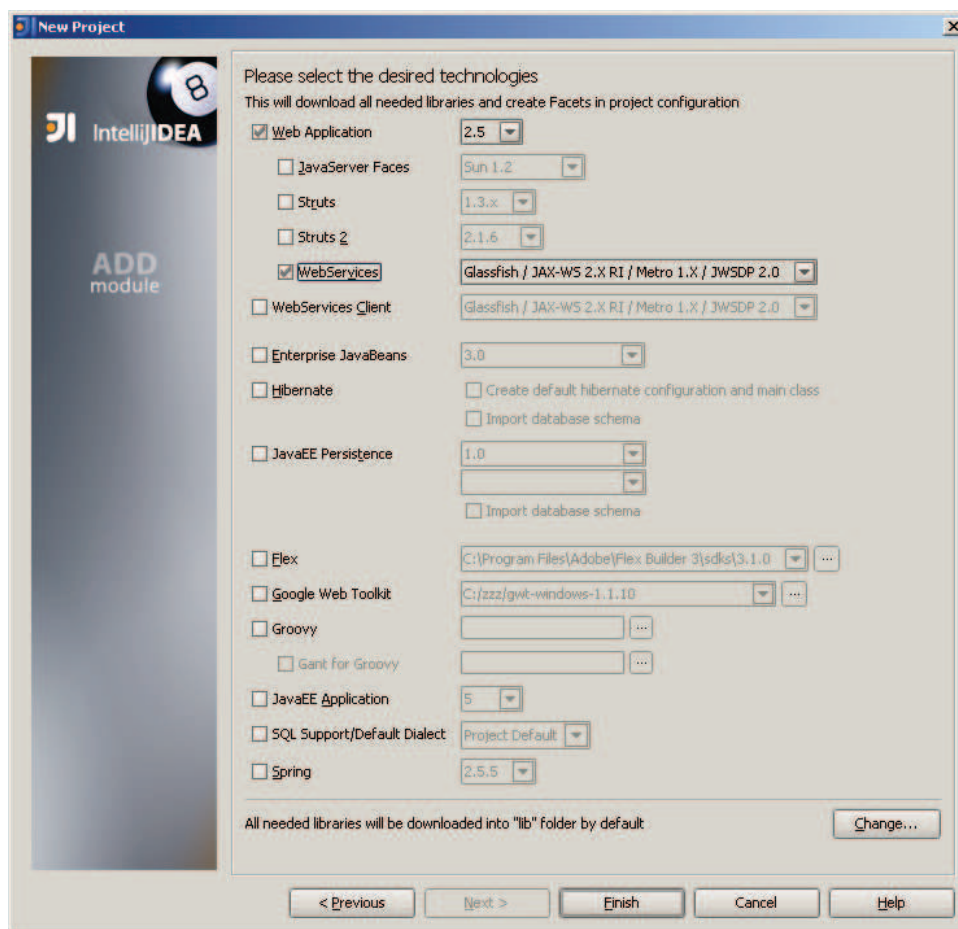
Well done, we have just created a small example of Web Service and its client application with Apache Axis framework.

Let's now have a look at a slightly different approach: JaxWS.

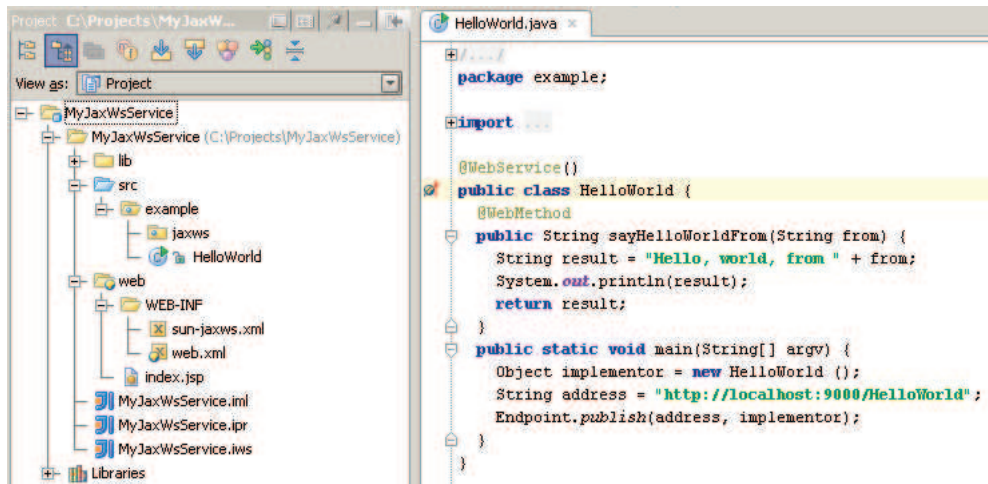
Creating Web Service: JaxWS

Creating JaxWS Web Services has much in common with the already described Apache Axis procedure, so we'll go into details only if there is a difference from the previous procedure.

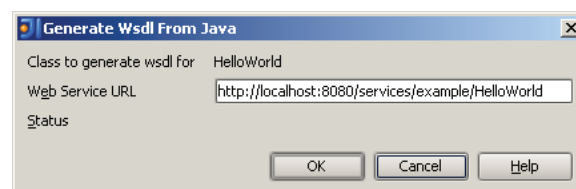
1. Run **IntelliJ IDEA** and choose **File | New Project**.
2. Specify the project name and location.
3. From the list of technologies, select **Web Application**, then **WebServices**, and select **Glassfish/JAX-WS 2.X RI/Metro 1.X/JWSDP 2.0**. Click **Finish**.



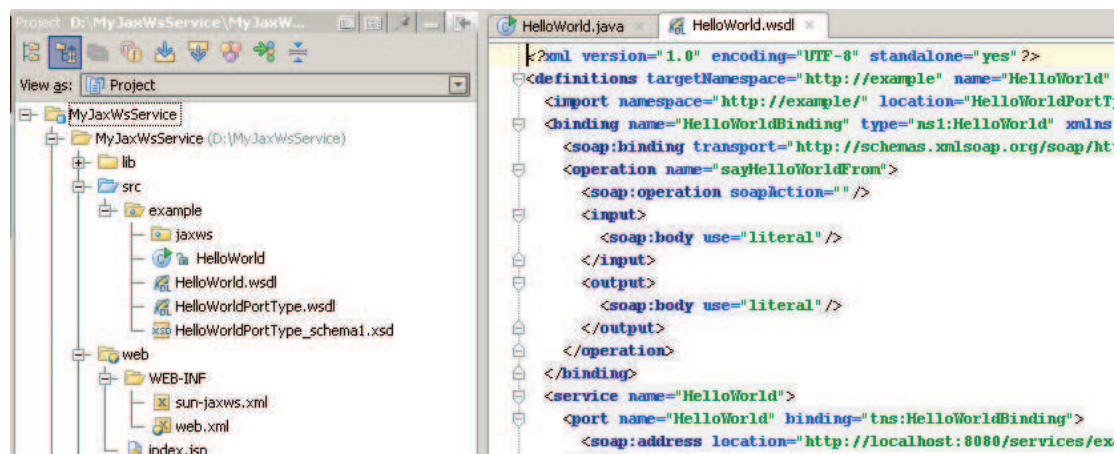
After IntelliJ IDEA creates the project, the sample code looks slightly different, because JaxWS relies on annotations instead of XML descriptors.



To generate WSDL descriptor right-click the class name in the editor and select **WebServices | Generate Wsdl From Java Code**.

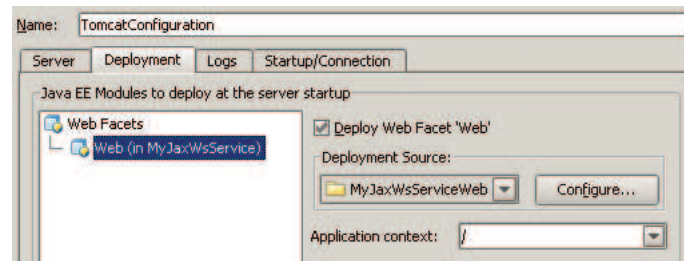


As you can see, we need only to specify the URL for the WebService and click OK. The descriptor is generated.

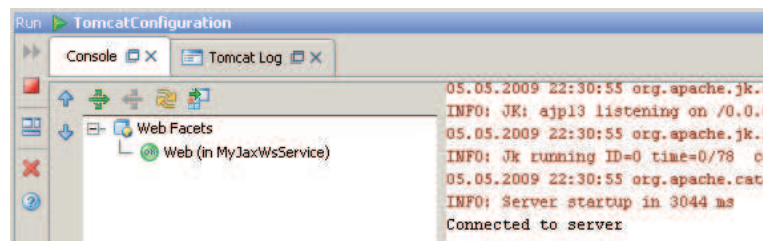


Two things are left to do to run our WebService.

1. Create Tomcat run/debug configuration like you did for the WebService with Apache Axis, and do not forget to set the **Deploy Web Facet 'Web'** option at the Deployment tab of the run/debug configuration.



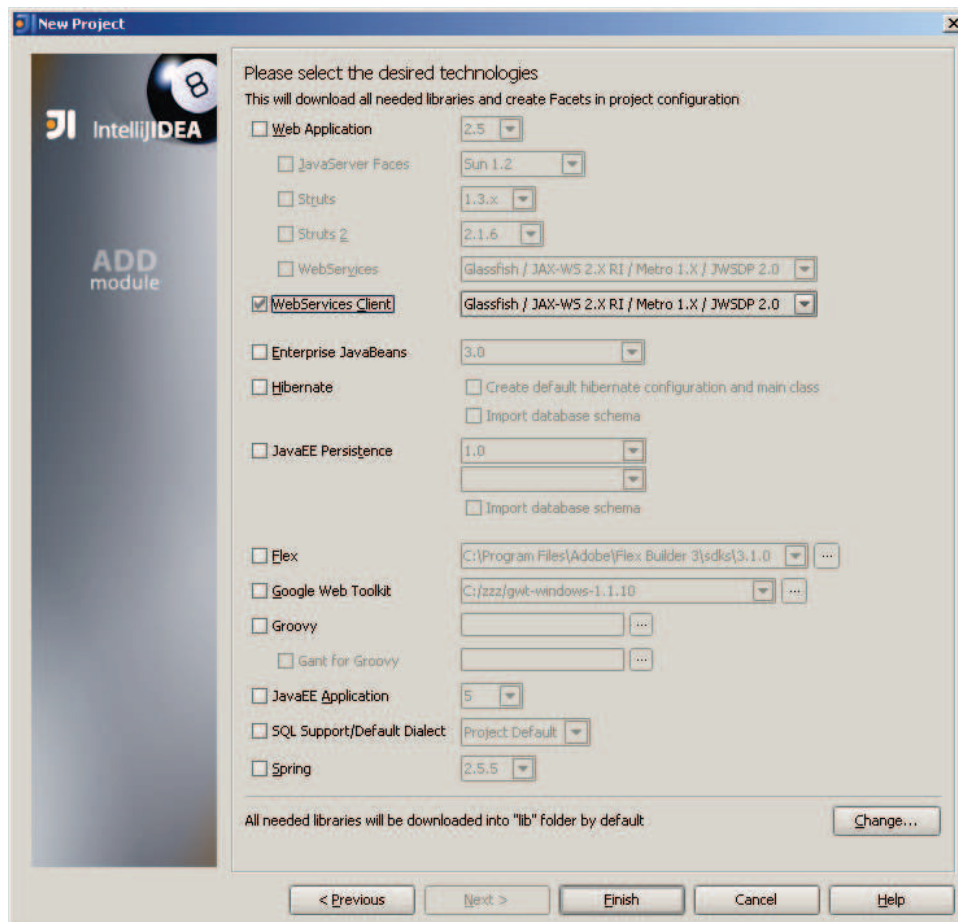
2. Press **SHIFT+F10** to run the WebService.



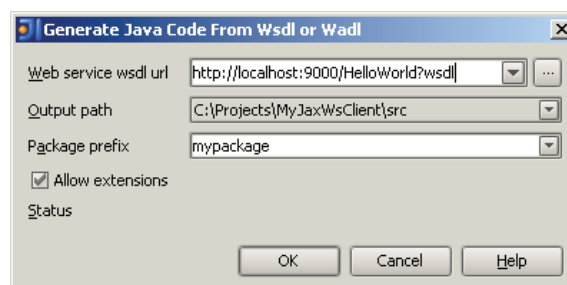
Creating Web Service Client: JaxWS

Client part for JaxWS Web Service is also a little different from that of Axis.

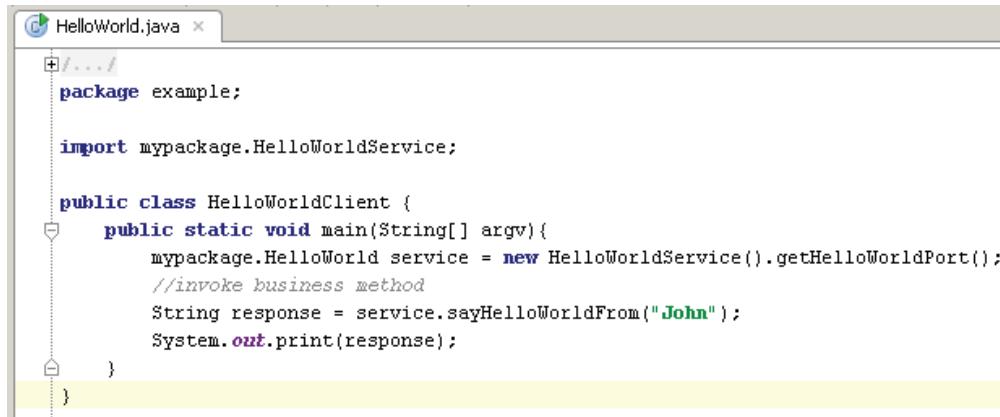
1. When creating the project, select **Glassfish/JAX-WS 2.X RI/Metro 1.X/JWSDP 2.0**



2. Generate code dialog requires fewer settings — basically you only need to type WSDL URL



3. After the code is generated, modify it so that the client is properly initialized and sends some data to service.

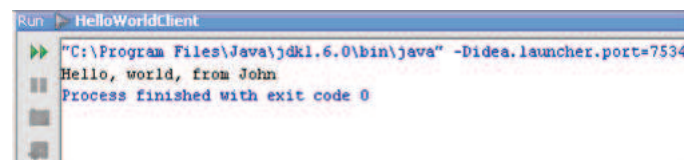


```
package example;

import mypackage.HelloWorldService;

public class HelloWorldClient {
    public static void main(String[] argv){
        mypackage.HelloWorld service = new HelloWorldService().getHelloWorldPort();
        //invoke business method
        String response = service.sayHelloWorldFrom("John");
        System.out.print(response);
    }
}
```

4. Now, press **CTRL+SHIFT+F10** to run the client. The response is printed in both consoles.

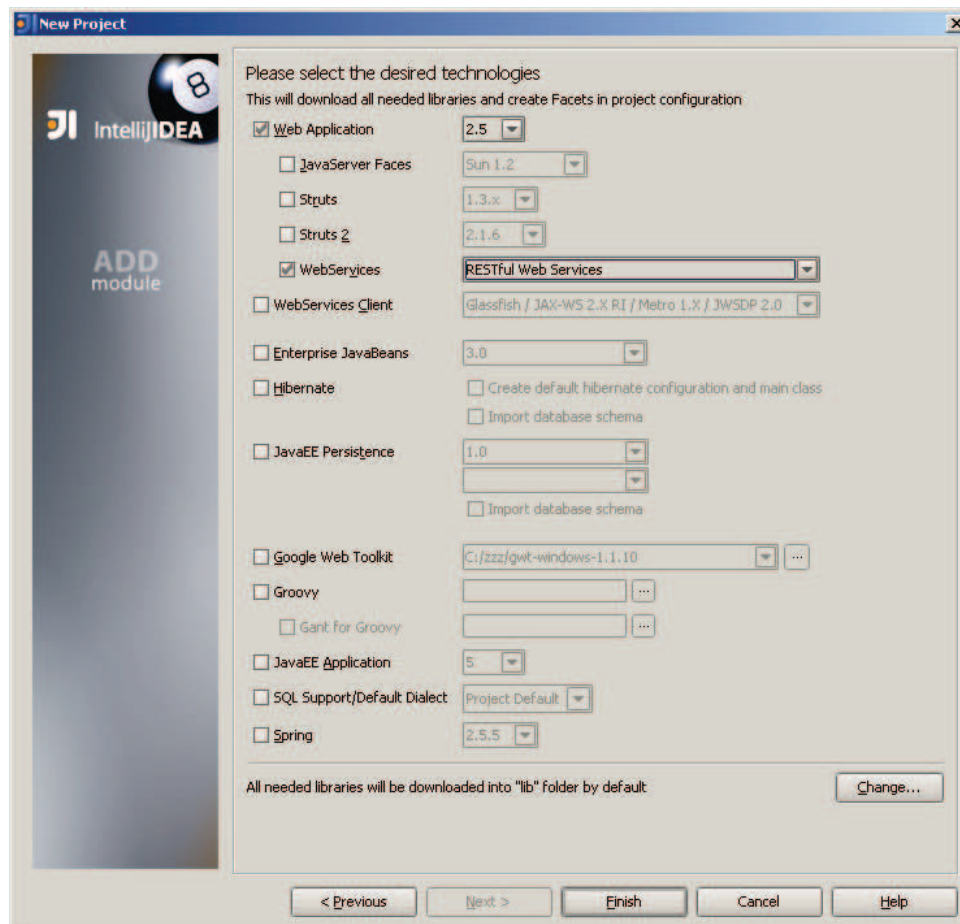


```
Run: HelloWorldClient
"C:\Program Files\Java\jdk1.6.0\bin\java" -Didea.launcher.port=7534
Hello, world, from John
Process finished with exit code 0
```

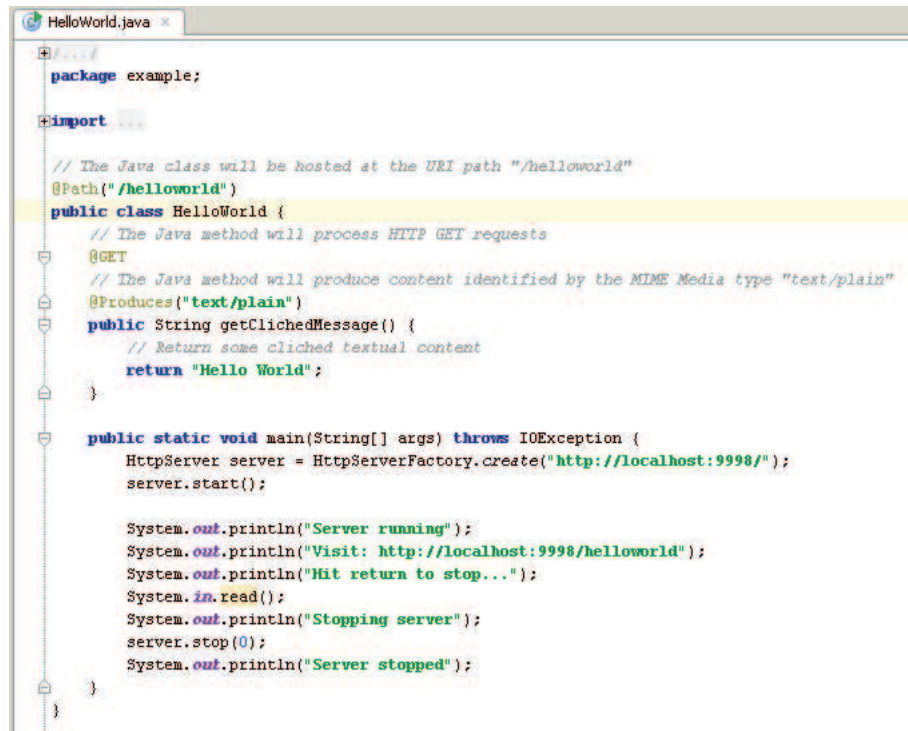
Creating Web Service: RESTful Web Services

We've already created sample projects using Apache Axis and JAX-WS. Now, it's time to try RESTful Web Services.

1. Run **IntelliJ IDEA** and start creating new project. On the technologies page of the **New Project Wizard** select **Web Application | WebServices | RESTful Web Services**.



The project structure is already familiar to you, but the HelloWorld class looks a bit different.



```

HelloWorld.java
+//...
package example;

import ...

// The Java class will be hosted at the URI path "/helloworld"
@Path("/helloworld")
public class HelloWorld {
    // The Java method will process HTTP GET requests
    @GET
    // The Java method will produce content identified by the MIME Media type "text/plain"
    @Produces("text/plain")
    public String getClichedMessage() {
        // Return some cliched textual content
        return "Hello World";
    }

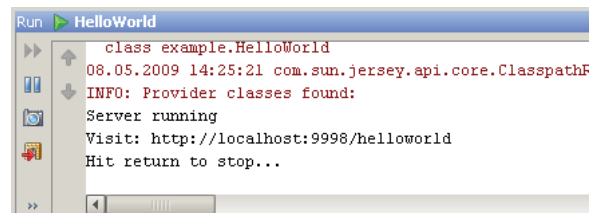
    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServerFactory.create("http://localhost:9998/");
        server.start();

        System.out.println("Server running");
        System.out.println("Visit: http://localhost:9998/helloworld");
        System.out.println("Hit return to stop...");
        System.in.read();
        System.out.println("Stopping server");
        server.stop(0);
        System.out.println("Server stopped");
    }
}

```

In contrast to previous examples, we can run our web service right at this step, i.e. we do not need Tomcat configuration, the service can be launched using light-weight HTTP server.

Just right-click anywhere in the HelloWorld class code and select **Run "HelloWorld.main()"** from editor popup menu, or press **Ctrl+Shift+F10**. IntelliJ IDEA creates temporary run/debug configuration that you can save, if you want, and runs the service.



```

Run HelloWorld
>> class example.HelloWorld
08.05.2009 14:25:21 com.sun.jersey.api.core.ClasspathF
INFO: Provider classes found:
Server running
Visit: http://localhost:9998/helloworld
Hit return to stop...
>>

```

Note: IntelliJ IDEA provides convenient tool window for testing RESTful web services with GET, POST, DELETE, PUT, HEAD and OPTIONS requests. Start debugging your service (right-click anywhere in the HelloWorld class code and select **Debug "HelloWorld.main()"**), then select **Tools | WebServices | RESTful Web Services | Test RESTful Web Service** from the main menu to open the **REST Client** tool window, where you can easily browse the results of GET, POST, DELETE, PUT, HEAD and OPTIONS requests.

Before we pass on to creating RESTful client, we need to get WADL descriptor, and here is another trick. We have two ways of generating WADL:

1. The first one is similar to generating WSDL. Right-click the class name in the editor and select **WebServices | RESTful Web Services | Generate WADL From Java Code**.

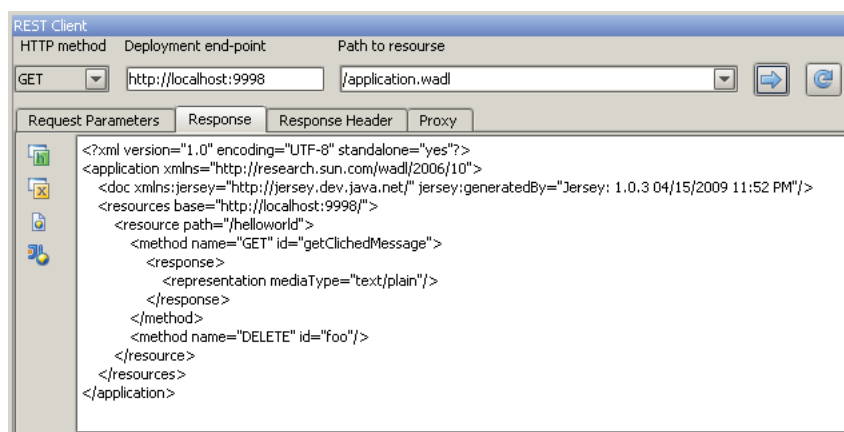


Specify **Base URI** and click OK.

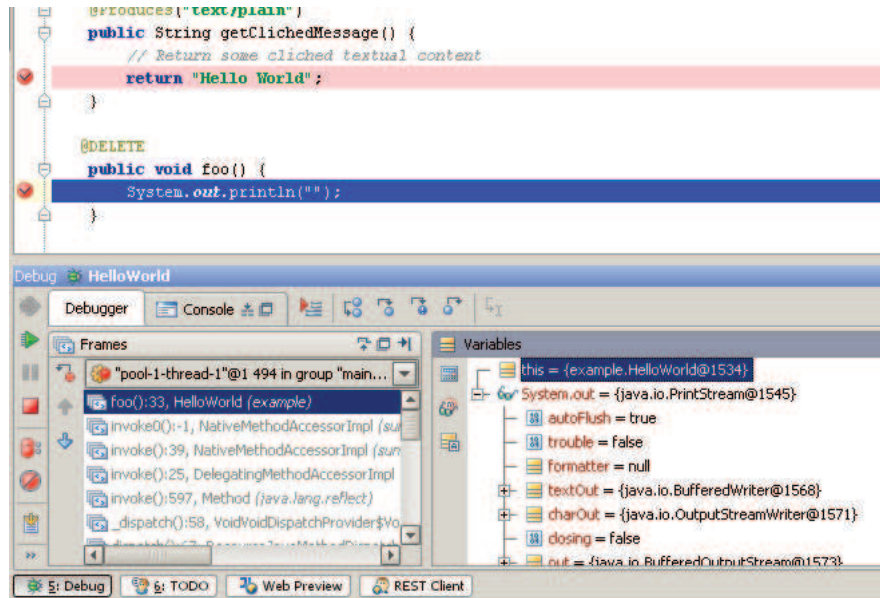


2. For the second one we'll need REST Client tool window.

While the service is running, right-click anywhere in the HelloWorld class in the editor, and select **WebServices | RESTful Web Services | Test RESTful Web Service** to access the tool window. Select GET from the **HTTP method** drop-down. Leave the **Deployment end-point** field as is, in the **Path to resource** drop-down select **/application**. **wadl**, then click **Submit request** button (). IntelliJ IDEA generates WADL descriptor, and displays it in the **Response** tab.



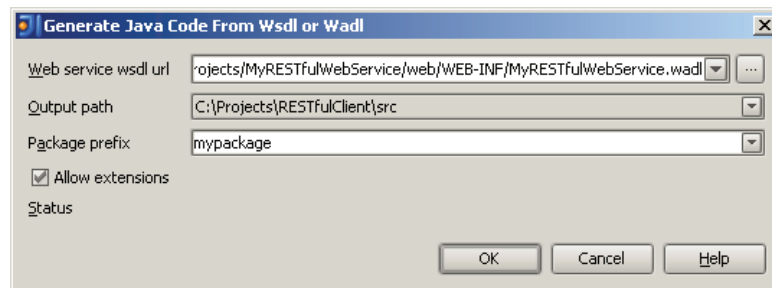
You can also debug your RESTful web service with the help of REST Client, which is very handy, when you need to examine DELETE or PUT requests, for example.



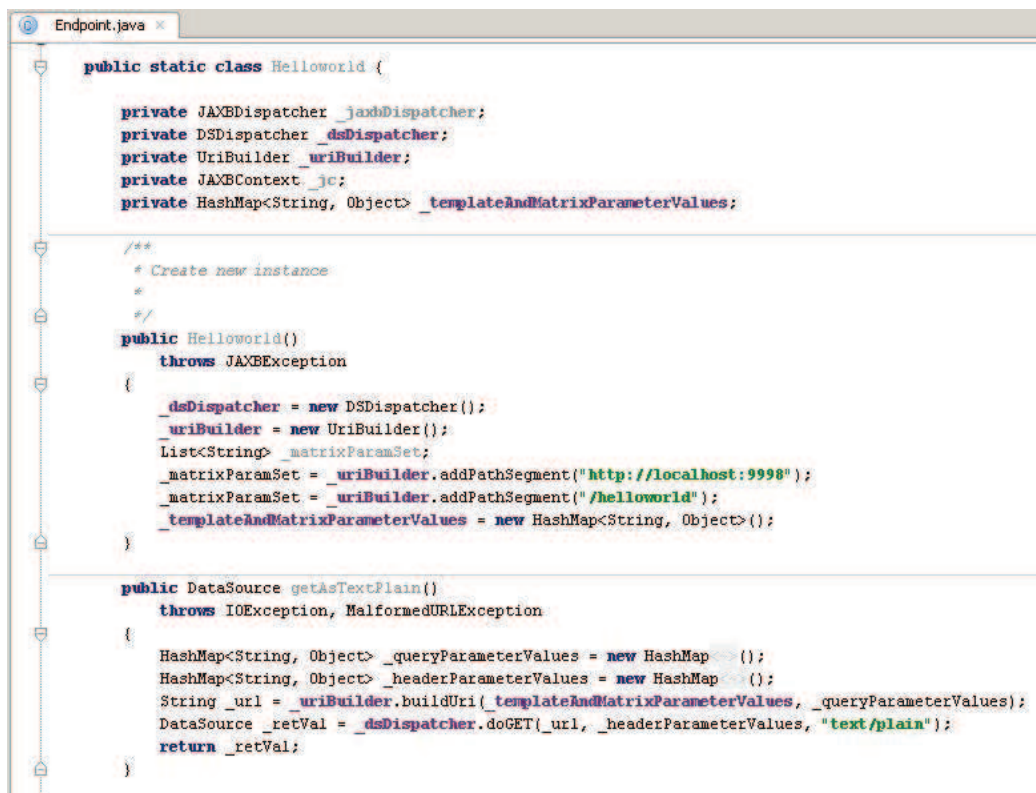
Creating Web Service: RESTful Web Services Client

The client part creation is also extremely similar to the ones created in this tutorial

1. Create new project. On the technologies page, select **Web Services Client | RESTful Web Services**. Click **Finish**.
2. In the dialog that appears specify the url to the generated WADL, or path to the locally stored WADL.



3. Client code is generated automatically.



That's all for now. Of course, this tutorial only covers the essential topics you will need to build a Web Service application with IntelliJ IDEA.

There's a lot more of things to explore and features that can help you enhance your application: integration with Hibernate, Spring, support for EJB, JSP, AJAX, integration with many popular application servers, and a whole lot more. Check out <http://www.jetbrains.com/idea> for additional details.